Practical : 11	Implementation of Floyd-Warshall Algorithm
Date : __ / __ / 2026	

Aim: To implement the **Floyd-Warshall algorithm** in C++ to find the **shortest paths between all pairs of vertices** in a weighted directed graph.

Objective :- To store a weighted directed graph as a distance matrix, apply dynamic programming to compute all-pairs shortest paths, detect negative cycles, and analyze the time and space complexity.

A. Graph Representation (Distance Matrix)

1. Start.
2. Read the number of vertices V and the number of edges E.
3. Create a V x V matrix dist[][]; set dist[i][i] = 0 and every other entry to INF (no path).
4. For each directed edge (u, v, w), set dist[u][v] = w.
5. Display the initial distance matrix.
6. Stop.

B. Floyd-Warshall Algorithm

1. Start.
2. Take the distance matrix dist[][] built in Part A.
3. For every intermediate vertex k from 0 to V - 1:
4. For every source vertex i and every destination vertex j:
5. If dist[i][k] and dist[k][j] are not INF and dist[i][k] + dist[k][j] < dist[i][j], set dist[i][j] = dist[i][k] + dist[k][j].
6. After all values of k are processed, if any dist[i][i] < 0, the graph has a negative cycle.
7. Display the shortest distance matrix (INF means there is no path).
8. Stop.

Program A: Floyd-Warshall Algorithm (practical11_floyd.cpp)

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;
const int INF = 1000000000;    // represents "no path"
class Graph {
    int V;
    vector<vector<int>> dist;
public:
    Graph(int vertices) {
        V = vertices;
        dist.assign(V, vector<int>(V, INF));
        for (int i = 0; i < V; i++) dist[i][i] = 0;
    }
    // Add a directed weighted edge u -> v
    void addEdge(int u, int v, int w) {
        dist[u][v] = w;
    }
    // Display a matrix (INF shown for "no path")
    void displayMatrix(const string &title) {
        cout << "\n--- " << title << " ---" << endl;
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][j] == INF)
                    cout << setw(5) << "INF";
                else
                    cout << setw(5) << dist[i][j];
            }
            cout << endl;
        }
    }
    // Floyd-Warshall Algorithm (all pairs shortest paths)
    bool floydWarshall() {
        for (int k = 0; k < V; k++) {
            for (int i = 0; i < V; i++) {
                for (int j = 0; j < V; j++) {
                    if (dist[i][k] != INF && dist[k][j] != INF &&
                        dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }
        // A negative value on the diagonal means a negative cycle exists
        for (int i = 0; i < V; i++) {
            if (dist[i][i] < 0) return false;
        }
        return true;
    }
};
int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;
    cout << "Enter number of edges: ";
    cin >> E;
    Graph g(V);
    cout << "Enter edges (u v weight):" << endl;
    for (int i = 0; i < E; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u < 0 || u >= V || v < 0 || v >= V) {
            cout << "Invalid edge. Vertices must be between 0 and "
                 << V - 1 << "." << endl;
            i--;
            continue;
        }
        g.addEdge(u, v, w);
```

```
    }
    g.displayMatrix("Initial Distance Matrix");
    bool ok = g.floydWarshall();
    g.displayMatrix("Shortest Distance Matrix");
    if (!ok) {
        cout << "\nGraph contains a negative weight cycle." << endl;
    }
    return 0;
}
```

Output:

Sample graph used: 4 vertices (0 to 3) and 4 directed edges (u to v: weight): 0 to 1: 5, 0 to 3: 10, 1 to 2: 3, 2 to 3: 1. The shortest path from 0 to 3 costs 9 (0, 1, 2, 3), which is less than the direct edge of 10.

```
Output

Enter number of vertices: 4
Enter number of edges: 4
Enter edges (u v weight):
0 1 5
0 3 10
1 2 3
2 3 1

--- Initial Distance Matrix ---
    0    5  INF   10
  INF    0    3  INF
  INF  INF    0    1
  INF  INF  INF    0

--- Shortest Distance Matrix ---
    0    5    8    9
  INF    0    3    4
  INF  INF    0    1
  INF  INF  INF    0


=== Code Execution Successful ===
```

Program B: Performance Analysis (practical11_analysis.cpp)

```
#include <iostream>
#include <vector>
#include <random>
#include <climits>
#include <algorithm>
#include <chrono>
using namespace std;
using namespace chrono;
const int INF = 1000000000;
// Floyd-Warshall Algorithm (all pairs shortest paths)
void floydWarshall(vector<vector<int>> &dist, int V) {
    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != INF && dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}
```

```
// Build a random directed graph with V vertices and E edges (fixed seed)
vector<vector<int>> generateGraph(int V, int E) {
    mt19937 rng(42);
    vector<vector<int>> dist(V, vector<int>(V, INF));
    for (int i = 0; i < V; i++) dist[i][i] = 0;
    for (int i = 0; i < E; i++) {
        int u = rng() % V, v = rng() % V;
        if (u == v) { i--; continue; }
        dist[u][v] = rng() % 100 + 1;
    }
    return dist;
}
int main() {
    int sizes[] = {10, 50, 100, 200, 300, 400, 500};
    cout << "Floyd-Warshall Algorithm Performance Analysis" << endl << endl;
    for (int V : sizes) {
        int E = 5 * V;
        vector<vector<int>> original = generateGraph(V, E);
        vector<vector<int>> dist;
        long long us = LLONG_MAX;
        for (int run = 0; run < 3; run++) {          // best of 3 runs
            dist = original;                         // fresh copy (not timed)
            auto start = high_resolution_clock::now();
            floydWarshall(dist, V);
            auto stop = high_resolution_clock::now();
            us = min(us, (long long)duration_cast<microseconds>(stop - start).count());
        }
        cout << "Number of Vertices = " << V << endl;
        cout << "Number of Edges    = " << E << endl;
        cout << "Shortest Distance  : ";
        if (dist[0][V - 1] == INF) cout << "No path from 0 to " << V - 1 << endl;
        else cout << dist[0][V - 1] << " (0 to " << V - 1 << ")" << endl;
        cout << "Floyd Time         : " << us << " microseconds" << endl;
        cout << "----------------------------------------" << endl;
    }
    return 0;
}
```

Output:

Random directed graphs with V vertices and E = 5V edges (weights 1 to 100) were used for V = 10, 50, 100, 200, 300, 400 and 500. Each time is the best of 3 runs.

```
Floyd-Warshall Algorithm Performance Analysis

Number of Vertices = 10
Number of Edges    = 50
Shortest Distance  : 21 (0 to 9)
Floyd Time         : 14 microseconds
---------------------------------------
Number of Vertices = 50
Number of Edges    = 250
Shortest Distance  : 77 (0 to 49)
Floyd Time         : 1311 microseconds
---------------------------------------
Number of Vertices = 100
Number of Edges    = 500
Shortest Distance  : 76 (0 to 99)
Floyd Time         : 9963 microseconds
---------------------------------------
Number of Vertices = 200
Number of Edges    = 1000
Shortest Distance  : 106 (0 to 199)
Floyd Time         : 73710 microseconds
---------------------------------------
Number of Vertices = 300
Number of Edges    = 1500
Shortest Distance  : 102 (0 to 299)
Floyd Time         : 246437 microseconds
---------------------------------------
Number of Vertices = 400
Number of Edges    = 2000
Shortest Distance  : 123 (0 to 399)
Floyd Time         : 579313 microseconds
---------------------------------------
Number of Vertices = 500
Number of Edges    = 2500
Shortest Distance  : 182 (0 to 499)
Floyd Time         : 1152515 microseconds
---------------------------------------
```

Observation:

When V doubled from 200 to 400, the time grew from 73710 to 579313 microseconds, about 7.9 times, which is close to $2^3 = 8$ expected from $O(V^3)$. Similarly, when V became 5 times larger (100 to 500), the time grew about 116 times ($5^3 = 125$).

The three nested loops always run V x V x V times, so the time depends on V and not on the number of edges.

The exact timings depend on the computer and compiler, so a different machine will show different numbers, but the growth pattern stays the same.

Analysis:

Time Complexity Analysis

Let V be the number of vertices of the graph.

Floyd-Warshall Algorithm:

There are three nested loops (k, i and j), and each one runs V times.

Every update is a single addition and comparison, which takes O(1) time. The running time does not depend on the number of edges.

Time Complexity = $O(V^3)$

Space Complexity Analysis

The distance matrix dist[][] has V x V entries. The updates are done in place, so no extra 3-D table is needed.

Space Complexity = $O(V^2)$

Note: after the three loops finish, a negative value on the diagonal dist[i][i] shows that a negative weight cycle exists, and then the shortest distances are not meaningful.

Comparison of All-Pairs Shortest Path Methods

Feature	Floyd-Warshall	Dijkstra (run V times)	Bellman-Ford (run V times)
Technique	Dynamic programming	Greedy + min-heap	Edge relaxation
Time complexity	$O(V^3)$	O(V (V + E) log V)	$O(V^2 E)$
Negative edges	Allowed	Not allowed	Allowed
Negative cycle detection	Yes (dist[i][i] < 0)	No	Yes
Best suited for	Dense graphs, small V	Sparse graphs, non-negative weights	Sparse graphs with negative edges

Conclusion :-

The Floyd-Warshall algorithm is a dynamic programming method that finds the shortest distance between every pair of vertices by allowing one more intermediate vertex at a time:

1. The program uses a **distance matrix** and three nested loops, giving **$O(V^3)$** time and **$O(V^2)$** space.

2. For the sample graph it correctly finds that the shortest path from vertex 0 to vertex 3 costs **9** (through vertices 1 and 2), and it shows INF where no path exists.

3. It works with negative edge weights and can detect negative cycles, but because of the cubic time it is suitable only for small or dense graphs.